Milestone 1 Report: Project Proposal, Requirements & Feasibility

Course Code:	CSET400C (Capstone Project)	**Academic Session:**	2026–27 (Semester VII)
Project Title:	**Elythra: AI Resume Screening & Ranking System**	**Assessment / Marks:**	Milestone 1 (Weightage: 30 Marks)
Team Leader:	Amandeep Verma (B.Tech CSE)	**Course Coordinator:**	Dr. Jabir Ali (Associate Professor)

1. Final Problem Statement and Motivation

Problem Statement: Companies and recruitment teams regularly receive hundreds of resumes for a single job opening. Manually reviewing each application against a job description is slow, tiring, and prone to human oversight. Most existing Applicant Tracking Systems (ATS) use simple keyword searches (Ctrl+F). This causes two major issues: qualified candidates are rejected just because they used different wording or a non-standard layout (false negatives), while candidates who simply repeat keywords from the job description get shortlisted without real skills (false positives).

Motivation: Recruiters typically spend only 6 to 10 seconds glancing over a resume during the first round. This quick scanning often introduces unintentional bias based on a candidate's name, gender, college name, or resume design. The goal of **Elythra** is to build an objective, automated tool that evaluates applicants on actual qualifications—skill overlap, semantic similarity, work experience, and education—helping recruiters shortlist candidates fairly and quickly without manual review fatigue.

2. UN Sustainable Development Goals (SDGs) Mapping & Rationale

UN Goal & Target	How This Project Directly Relates
SDG 8: Decent Work & Economic Growth **Target 8.5 (Productive employment for all)**	**Improving Job Matching Efficiency:** Automating resume screening cuts down the multi-week delay in hiring. Positions get filled faster with candidates who actually fit the role, reducing hiring costs and helping businesses and job seekers connect more efficiently.
SDG 10: Reduced Inequalities **Target 10.2 & 10.3 (Equal opportunity)**	**Removing Demographic & Pedigree Bias:** The system evaluates candidates strictly on verified skills and domain relevance. By stripping personal contact details and college names during scoring, every applicant gets judged on merit rather than background or gender.

3. Background and Context

With online job platforms like LinkedIn and Naukri, submitting applications has become as easy as clicking a button. As a result, a single corporate opening often receives 250 to 500 resumes within a few days. HR teams spend over 60% of their hiring time on initial resume filtering. The average corporate hiring cycle takes over 40 days, and costs around $4,000 per hire. Meanwhile, many job applicants never receive any feedback because basic keyword filters silently discarded their resume. An automated, skill-focused tool is needed to make this first step faster, transparent, and fair.

4. Literature & Existing-Solution Review

We reviewed both academic research and existing commercial tools to analyze how resume screening is handled currently:

• **Academic Research:** *Sinha et al. (2021)* showed that TF-IDF with n-grams offers fast, deterministic document similarity, though it requires a taxonomy to properly recognize synonyms. *Roy et al. (2020)* demonstrated rule-based NER for extracting contact info and experience. *Raghavan et al. (2020)* analyzed algorithmic hiring systems and confirmed that unconstrained machine learning models replicate historical human biases unless explicit anonymization is enforced.

• **Legacy Enterprise ATS (Oracle Taleo, Workday Recruiting):** Widely used by Fortune 500 corporations, but rely on strict exact-match keyword searches. They discard qualified candidates who use synonyms (e.g. 'React' vs 'React.js') and are vulnerable to 'keyword stuffing' tricks.

• **Modern Enterprise AI (Eightfold.ai, HireVue):** Use proprietary deep neural networks. While powerful, they function as 'black boxes' (failing transparency requirements under EU AI Act and NYC Local Law 144) and have prohibitive licensing fees ($25,000+/year) inaccessible to SMEs.

• **Candidate-Facing Scanners (Jobscan, SkillSyncer):** Single-resume checkers designed for job seekers. They count raw keyword frequency and cannot rank a batch of 50+ candidate resumes for a recruiter.

• **Open-Source Python Parsers (PyResparser):** Libraries like pyresparser extract contact info and skills, but do not match or rank resumes against a Job Description, and break on modern Python 3.10+.

• **Historical Case Study (Amazon's Scrapped AI Tool, 2018):** Amazon built an experimental machine learning tool that taught itself to penalize female candidates because it was trained on 10 years of male-dominated resumes. This proved why unconstrained deep learning on historical data is dangerous for hiring.

5. Comprehensive Gap Analysis of Existing Tools vs. Elythra

To clearly establish why our project is needed, the table below compares existing commercial tools, open-source libraries, and historical case studies against Elythra:

Existing Tool / System	Tool Category	Core Limitations / Flaws	How Elythra Solves This Problem
Oracle Taleo / Workday	Legacy Corporate ATS	• Rigid exact-match keyword searching (Ctrl+F). • Rejects synonyms (e.g., 'ML' vs 'Machine Learning'). • Easily gamed by candidates stuffing repeated keywords.	• Uses a **500+ Skill Taxonomy** linking synonyms. • Combines **35% TF-IDF semantic context** and experience so raw keyword repetition cannot game the score.
Eightfold.ai / HireVue	Modern Enterprise AI Platforms	• **'Black-Box' Opacity:** Scores cannot be explained. • Very expensive ($25k–$50k/year) for SMEs. • Legal risk under EU AI Act & NYC Law 144.	• **100% Explainable 4-Factor Matrix:** Shows exact breakdown of Skills (45%), Semantics (35%), Exp (10%), Edu (10%). • Built on **free, open-source Python stack**.
Jobscan / SkillSyncer	Candidate-Facing Resume Matchers	• Evaluates only **1 resume at a time** for job seekers. • Counts raw word frequency rather than experience. • No recruiter dashboard or batch candidate leaderboard.	• Supports **Batch Ingestion** (processes 50+ resumes concurrently). • Provides interactive recruiter dashboard with Plotly inspection gauges and Excel export.
PyResparser	Open-Source Python Library	• Outdated dependencies; broken on Python 3.10+. • **Only extracts raw text**; cannot compare, match, or rank candidates against a Job Description.	• Complete **end-to-end pipeline**: Ingestion → Extraction → Weighted Matching → Web Dashboard.
Amazon AI Tool (Scrapped 2018)	Historical ML Case Study	• Trained on historical hiring data; model developed bias against female applicants (penalized words like 'women's').	• **No biased historical training data used.** • Explicitly strips candidate names, gender, contact details, and college names before scoring.

6. Stakeholder & Requirements Analysis

Stakeholders: Recruiters who need fast initial shortlists; Hiring Managers who want to see candidate skill gaps; Job Seekers who want fair evaluation; and Organizations that need consistent hiring records.

Functional Requirements (FR):

- **FR-1: File Support:** Ingest resumes in PDF, DOCX, and TXT formats and clean up raw text.
- **FR-2: Skill Extraction:** Match extracted words against a curated list of 500+ technical skills categorized into 10 domains.
- **FR-3: Experience & Degree Extraction:** Extract total years of experience and educational qualifications (B.Tech, M.Tech, etc.) using regex.
- **FR-4: Multi-Factor Scoring:** Calculate a total score out of 100 based on Skill Overlap (45%), TF-IDF Similarity (35%), Experience Match (10%), and Education Match (10%).
- **FR-5: Skill Gap Tagging:** Clearly list which required skills the candidate has (Matched) and which ones are missing (Missing).
- **FR-6: Interactive UI & Reports:** Display ranked candidates on a Streamlit dashboard with score filters, candidate inspection cards, and 1-click export to CSV and Excel.

Non-Functional Requirements (NFR):

- **NFR-1 (Speed):** Process and score each resume in less than 1.5 seconds on standard laptop hardware without needing a GPU.
- **NFR-2 (Explainability):** Show the individual score components for every applicant so recruiters know why a score was given.
- **NFR-3 (Robustness):** Handle corrupt files or unusual formatting without crashing the entire batch.
- **NFR-4 (Fairness):** Exclude contact information, names, and personal details from the similarity calculation.

7. Technology/Tool Selection with Architectural Justification

The project follows a decoupled, modular architecture separated into Frontend, Backend, NLP/Machine Learning, and Data subsystems:

Architectural Layer / Subsystem	Technologies & Libraries	Role & Justification in Elythra
Frontend Layer (Recruiter UI & Visuals)	**Streamlit (v1.35+)** HTML5 & Custom CSS **Plotly (Graph Objects)** ReportLab	• **Streamlit:** Powers reactive web dashboard, multi-file uploaders, score sliders, and candidate cards without heavy Node/React overhead. • **HTML/CSS:** Styles custom gradient skill badges (green=matched, red=missing). • **Plotly:** Renders dynamic Donut gauges and Radar skill charts. • **ReportLab:** Compiles downloadable PDF candidate summaries.
Backend & Logic Layer (Core Application Engine)	**Python 3.10+** Modular Pipeline (core/) Regular Expressions (re)	• **Python:** Core backend language for text mining and application orchestration. • **Modular Architecture:** Decouples authentication (auth.py), text cleaning (text_cleaner.py), extraction (extractor.py), and scoring (matcher.py). • **Regex Engine:** Robust pattern extraction for candidate emails, phone numbers, experience years (e.g. '3+ years'), and degree levels.
NLP & ML Engine (Text Mining & Similarity)	**Scikit-Learn** (TF-IDF & Cosine Sim) **Curated Skill Taxonomy**	• **TfidfVectorizer:** Converts resumes and job descriptions into unigram+bigram vectors for semantic context matching (35% weight). • **Cosine Similarity:** Deterministic mathematical metric computing angular similarity between vectors. • **500+ Skill Taxonomy:** In-memory dictionary linking synonyms (e.g., 'React' & 'ReactJS') for exact skill overlap (45% weight).
Document Ingestion (Multi-format File Parsing)	**PyPDF (v4.2+)** **python-docx (v1.1+)**	• Ingests standard resume formats (.pdf, .docx, .txt) without requiring external heavy OCR installations. • Extracts multi-column paragraph flows into sanitized in-memory text buffers.
Data Processing & Export (Analytics & Reporting)	**Pandas** **NumPy** **OpenPyXL**	• **Pandas & NumPy:** Vectorized data structures for fast batch leaderboard filtering, sorting, and score calculation. • **OpenPyXL:** Generates professional, styled Excel (.xlsx) reports with color-coded score columns for recruiter export.
Testing & DevOps (Quality & Deployment)	**Unittest** (test_pipeline.py) Git & GitHub Render Cloud	• **Unittest:** Automated regression tests verifying parser integrity, regex extraction, and scoring accuracy. • **Git/Render:** Version control and cloud hosting specification (render.yaml).

8. Feasibility Analysis (Technical, Economic, Operational, Legal/Ethical)

• **Technical Feasibility:** Fully feasible. All core modules (text extraction, taxonomy matching, TF-IDF calculation, and dashboard) have already been coded and verified using unit tests. It runs locally on standard laptops without GPU requirements.
• **Economic Feasibility:** Built entirely using open-source Python libraries. There are zero software license fees or recurring API charges. Can be hosted on free or low-cost cloud tiers.
• **Operational Feasibility:** Very easy for HR teams to adopt. Recruiters just upload resumes and a job description, adjust weights if needed, and export shortlisted candidates to Excel.
• **Legal & Ethical Feasibility:** The system does not make automated rejection decisions; it acts as a decision-support tool. It avoids bias by isolating personal identifiers and provides full score breakdowns for transparency.

9. Scope and Limitations

In Scope: Parsing standard text-based resumes (PDF, DOCX, TXT); extracting 500+ technical skills; calculating multi-factor scores; displaying an interactive ranking dashboard; visual matched vs. missing skills; 1-click CSV/Excel export; demo dataset for quick testing.
Limitations & Out of Scope: Scanned image PDFs that require OCR are not supported in this phase (planned for next milestone); only English-language resumes are parsed; does not include video/audio analysis.

10. Risk Analysis and Mitigation Strategy

Identified Risk	Severity / Likelihood	Impact	Mitigation Strategy
Keyword Stuffing	Medium / Medium	Candidates repeat buzzwords to rank higher.	Multi-factor weighting: TF-IDF context similarity and experience verification dilute raw keyword repetition.
Varied Resume Layouts	Medium / High	Multi-column designs can scramble extracted text.	Dual parsing logic with regex cleaners and fallback handling to ensure files don't crash the pipeline.
Bias from Demographics	High / Low	Unintended bias from candidate names or college tiers.	Personal identifiers (email, phone, links) are stripped before running similarity algorithms.
Slow Processing on Batches	Low / Low	Dashboard lag when uploading 50+ resumes at once.	Vectorized text processing via NumPy/Pandas and Streamlit caching (@st.cache_data) for fast execution.

11. Proposed Solution and Detailed Methodology

Proposed Solution: Elythra is a lightweight, Python-based resume screening and candidate ranking application. It takes a job description and a batch of resumes, parses their contents, identifies technical skills and qualifications, and computes an overall weighted compatibility score out of 100 using four clear components:

Total Score = (0.45 × Skill Overlap) + (0.35 × TF-IDF Cosine Similarity) + (0.10 × Experience Match) + (0.10 × Education Match)

Step-by-Step Engineering Methodology:

- **Step 1: Requirement Gathering & Dataset Setup:** Defined core input formats (.pdf, .docx, .txt), collected representative job descriptions (e.g., Python Backend Lead, Data Scientist), and compiled realistic test resumes spanning various skill levels.
- **Step 2: Document Parsing & Text Preprocessing:** Implemented modular parsers (parser.py) using pypdf and python-docx. Text is cleaned in text_cleaner.py by lowercasing, removing extra whitespace and special characters, and extracting contact details via regular expressions.
- **Step 3: Taxonomy Design & Feature Extraction:** Built a categorized dictionary of 500+ technical skills in skills_database.py covering Languages, Web, Backend, AI/ML, Cloud, Databases, and DevOps. Built extraction logic in extractor.py to identify skills, years of experience, and degrees.
- **Step 4: Scoring Engine Implementation:** Developed the matching engine in matcher.py. Skill overlap is calculated as the intersection over required skills. Semantic similarity is computed using Scikit-Learn's TfidfVectorizer (unigrams + bigrams) and cosine similarity. Experience and education are scored against job minimums.
- **Step 5: Web Dashboard Development:** Created an interactive recruiter interface using Streamlit (app.py). Displays KPI metrics, a ranked candidate leaderboard, Plotly gauge charts for individual candidate inspection, color-coded skill badges, and one-click CSV/Excel report exporting.
- **Step 6: Automated Testing & Validation:** Implemented end-to-end integration tests in test_pipeline.py to verify that parsing, extraction, and ranking algorithms execute accurately across all sample files.

12. Expected Outcomes & Project Timeline

- **Expected Outcomes:** A functional Streamlit web application; automated batch processing in <1.5s per resume; transparent candidate scoring with zero black-box confusion; visual skill-gap indicators; and clean CSV/Excel reports for hiring teams.
- **Capstone Evaluation Schedule:**
  - ✔ **Milestone 0 (25 Aug 2026):** Problem Identification & Group Formation *[Completed / Approved]*
  - ■ **Milestone 1 (18 Oct 2026):** Project Proposal, Requirements, Feasibility & Methodology *[Current Submission - 30 Marks]*
  - ■ **Milestone 2 (23 Nov 2026):** Design, Working MVP/Prototype & Initial Validation *[Upcoming - 30 Marks]*
  - ■ **Final Panel Evaluation (End-Semester):** Report, Presentation, Live Demo & Viva *[Upcoming - 30 Marks]*
  - ■ **Guide Evaluation (Continuous):** Individual Assessment & Contribution *[10 Marks]*

Submitted By (Team Leader):

Amandeep Verma (Enrollment No: Refer BU Records)
B.Tech CSE, School of Computer Science Engg. & Technology
Bennett University, Greater Noida

Course Coordinator Approval:

Dr. Jabir Ali (Course Coordinator, CSET400C)
Associate Professor, School of Computer Science Engg. & Technology
Bennett University, Greater Noida